Fleet Intelligence Agent — Fully Autonomous Execution

Engineer Implementation Guide

Scope: `fleet-svc` + `ai-orchestration-svc` (backend), `voyqi-fleet-management.html` (admin UI) **Status:** Complete. 887/887 backend tests passing, 72/72 frontend smoke-test assertions passing. **Migration head:** `f1a9c3d7e802`

---

1. Executive summary

The Fleet Intelligence Agent covers 13 capabilities across fleet operations (FLT-001 through FLT-013). Every one of them follows the same architecture: a deterministic decision function (or a live Claude call, if `ANTHROPIC_API_KEY` is configured) produces a verdict, and that verdict **is** the executed action — not a proposal awaiting sign-off. The agent's tool call and the resulting state change happen atomically, in the same request.

Twelve of the thirteen capabilities have no human gate at all. The one exception, documented explicitly in §4, is unfreezing a floorplan facility after a covenant breach — clearing that requires a human attestation the agent doesn't have standing to make on its own.

This document describes the system as implemented: what each capability decides, what it executes, where the code lives, how the capabilities chain into each other, and how to verify all of it.

2. Design principles

- **The agent's output is the action.** A capability that only writes an advisory note isn't finished — it should mutate the state the note is about, in the same call.
- **Structural boundaries stay structural, not procedural.** Fields like `liability_assessment` (incident fault), `is_covenant_breached` and every dollar figure on a floorplan facility, and VIN collateral status are never touched by any agent function — not gated behind a check, simply absent from the code path entirely. This is stronger than a "human should review this" comment, and it's verifiable by tests (e.g. a test asserting a function never calls anything liability-related).
- **One coarse kill switch, no per-capability tiers.** `Agent.status` (`ACTIVE` / `PAUSED` / `DISABLED`) is the only operational lever. There is no tiering config that lets one capability run autonomously while another requires approval — if you need that, it's a separate piece of work layered on top of this one, not present here.

- **Chains fail closed on their own step, not on their parent.** When one capability's outcome triggers another (see §5), the trigger call is isolated so a failure in the downstream step never corrupts or blocks the upstream decision that already committed.

3. The kill switch

`Agent.status` (`ACTIVE` / `PAUSED` / `DISABLED`) on the `agents` table remains the one operational safety valve. Every `evaluate_*` function in `fleet_intelligence_agent.py` checks this first:

```
status = await _agent_status(db, tenant_id)
if status != AgentStatus.ACTIVE:
    log = DecisionLog(..., action_taken="skipped_agent_not_active",
outcome="deferred")
    ...
    return SomeResult(decision_log=log, skipped_reason=status.value)
```

If an agent is `PAUSED`, none of the autonomous actions in this doc fire. This is whole-agent on/off, not a per-capability tier system — there is no `tier-policy.yaml` in this codebase. If you need per-capability tiering, that's separate scope, not part of this change.

---

4. Per-capability reference

For each capability: **what the agent decides**, **what it now executes**, **which file/function**, and **any chain to another capability**.

FLT-001 — Intake Readiness

- **Decision:** `provision_ready` / `provision_with_flag` / `hold_for_review`, from a keyword scan of inspection notes (`inventory_engine.py`, deterministic fallback) or live Claude.
- **Autonomous action:**
  - `hold_for_review` → `Vehicle.held_for_review = True`, `held_for_review_reason` set. Vehicle stays unprovisioned, blocked from staging.
  - `provision_ready` / `provision_with_flag` → auto-provisions (creates `ProvisioningRecord`, sets `is_provisioned = is_class_certified = True`, `status = STAGED`).
- **Function:** `inventory_engine.auto_execute_intake_readiness(db, tenant_id, vehicle_id, recommendation, reasoning, agent_key)`
- **Wired at:** `POST /api/v1/fleet/intake-inspections/{id}/readiness-assessments` (the same endpoint the agent's `record_intake_readiness` tool already called — recording and executing now happen in the same call).
- **Backend engine used by frontend too:** `deterministicIntakeReadiness(passed, notes)` in the HTML (mirrors `INTAKE_CONCERN_KEYWORDS` list exactly).

- **Important nuance:** `hold_for_review` only fires on a **failed** inspection (`passed=False`). A passed inspection with a concern keyword in the notes produces `provision_with_flag` (flags, doesn't block) — never `hold_for_review`. The UI's "Fail inspection" button exists specifically to make this branch reachable (`inspectPO(id, passed)` takes a `passed` argument, alongside "Inspect & pass").

FLT-002 — Telemetry / Geofence Anomaly

- **Decision:** `benign` / `needs_investigation`, from distance beyond the 50-mile geofence radius, versus a 150-mile investigation threshold.
- **Autonomous action:** `needs_investigation` → `Vehicle.status = MAINTENANCE`, `Vehicle.flagged_for_investigation = True`. `benign` → no action.
- **Function:** `telematics_engine.auto_execute_telemetry_anomaly(db, tenant_id, vehicle_id, assessment, agent_key)`
- **Wired at:** `POST /api/v1/fleet/geofence-alerts/{id}/anomaly-assessments`
- **Structural boundary:** tamper events (`record_tamper_event` / `reportTamper()`) never pass through this function at all — not gated, not routed through the AI, structurally separate.
- **Bug fixed during this work:** `telematics_engine.py` was missing the `VehicleStatus` import, which would have raised `NameError` the first time this code path executed. Caught by the fleet-svc test suite, fixed, re-verified.

FLT-003 — Staging & Hub Rebalancing

- **Decision:** whether a destination hub is short on a vehicle class; if so, which hub has surplus to pull from.
- **Autonomous action:** creates **and dispatches** the transfer in one call — `TransferOrder.status = DISPATCHED` directly (never `PROPOSED`), `dispatched_by = agent_key`, `driver_id = "auto-dispatch-logistics-queue"`, `Vehicle.status = IN_TRANSIT`.
- **Function:** `staging_engine.auto_dispatch_transfer(db, tenant_id, vehicle_id, to_hub_id, reason, agent_key)`
- **Wired at:** `POST /api/v1/fleet/transfer-orders/auto-dispatch` (a separate `POST /transfer-orders` + `POST /transfer-orders/{id}/dispatch` two-step flow exists for manual/non-agent use).
- **Agent tool:** `fleet_tools.auto_dispatch_transfer`. This is also the Anthropic tool-schema name the live-Claude path uses — anything wiring up external tooling or an eval harness against this agent should use this name.
- **Result fields:** `StagingRecommendationResult.dispatched_transfer` holds the transfer payload (or `None`); `DecisionLog.action_taken` is `"transfer_dispatched"` or `"no_transfer_needed"`.
- **Frontend:** `requestStagingRecommendation()` sets `status: 'dispatched'` and `vehicle.status = 'in_transit'` directly — there is no separate dispatch-click UI, since a transfer is never created in a state that needs one.

FLT-004 — Maintenance Triage

- **Decision:** priority order of open alerts, sorted by severity then DTC-code count. Runs once 3+ alerts are open (`MAINTENANCE_TRIAGE_THRESHOLD` / `TRIAGE_ALERT_COUNT_THRESHOLD`).
- **Autonomous action:** opens a real `WorkOrder` for the single top-priority alert (`vendor="auto-dispatch-queue"`), and acknowledges that alert so it drops out of the next triage batch. Lower-priority alerts stay open for the next accumulation.
- **Function:** `maintenance_engine.auto_dispatch_top_priority_work_order(db, tenant_id, priority_order, agent_key)`
- **Wired at:** `POST /api/v1/fleet/maintenance-triage-assessments`
- **Defensive detail:** this function validates `priority_order[0]` as a real UUID and looks it up before touching anything. A stale/placeholder/non-existent ID is skipped silently (returns `None`) rather than raising — the triage record itself is always written regardless of whether the top alert can be resolved to a real row.

FLT-005 — Incident Severity

- **Decision:** `minor` / `moderate` / `severe`, from a keyword scan of the incident description.
- **Autonomous action:** `severe` → opens a `RepairOrder` immediately (`vendor="pending-assignment"`, `estimated_cost_usd=0.0`), moving the incident into the `REPAIRING` pipeline. `minor`/`moderate` take no further action beyond the classification.
- **Function:** `incident_engine.auto_execute_incident_severity(db, tenant_id, incident_id, severity, agent_key)`
- **Wired at:** `POST /api/v1/fleet/incidents/{id}/severity-assessments`
- **Structural boundary:** this function has no path to `liability_assessment` or `assessed_by`. Liability is set exclusively by the separate, always-human `assess_liability` endpoint/function. This is verified by a test named `test_incident_severity_never_calls_any_liability_related_function`.
- **Frontend:** `reportIncident()` sets `incident.status = 'repairing'` and `incident.repair_vendor = 'pending-assignment'` when severity is `severe`. `renderIncidents()` shows the liability-assessment buttons whenever `!inc.liability_assessment` (rather than gating on `status === 'reported'`, which would hide them once status moves to `'repairing'`).

FLT-006 — Valuation Anomaly *(chains into FLT-013)*

- **Decision:** `normal` / `flagged`, from % deviation between a fresh market valuation and book value (±20% threshold, either direction).
- **Autonomous action:** never touches the depreciation schedule or valuation record, which stay deterministic arithmetic. Instead, `flagged` **immediately triggers a full FLT-013 fleet-optimization review** for the vehicle — see the chain diagram in §5.
- **Function:** `depreciation_engine.record_valuation_anomaly` records the assessment; the chain is published from `POST /api/v1/fleet/asset-valuations/{id}/anomaly-assessments`, which calls `analytics_engine.gather_fleet_optimization_signals(...)` and publishes a `flt.fleet_optimization_requested` Kafka event when the assessment is `FLAGGED`.

- **Response field added:** `chained_to_fleet_optimization: bool` on the response body, so callers can see whether the chain fired.

FLT-007 — Disposition Channel Cascade

- **Decision:** which cascade stage a disposition-eligible vehicle should move to — `rideshare` (<75k mi) → `local_rental` (75k–90k) → `export` (90k–120k) → `parts` (≥120k). Mileage-band boundaries are inclusive on the lower bound (`>= 75000` is `local_rental`, not `rideshare`).
- **Autonomous action:** creates a **real `DispositionOrder`** in the same call, `Vehicle.status = REMARKETING` immediately. The `DispositionChannelRecommendation` row is written as `ACCEPTED` (never `PENDING`) for a complete audit trail, but it's not a gate — the order already exists by the time the row is written.
- **Function:** `remarketing_engine.create_disposition_order_autonomous(db, tenant_id, vehicle_id, channel, reasoning, agent_key)`
- **Wired at:** `POST /api/v1/fleet/vehicles/{id}/disposition-recommendations` (same endpoint name as before; behavior changed underneath it)
- **Fallback path, not the primary flow:** `accept_disposition_recommendation` / `reject_disposition_recommendation` and their endpoints (`POST /disposition-recommendations/{id}/accept`, `.../reject`, `GET /disposition-recommendations/pending`) exist for one deliberate case — a `PAUSED` agent that wants to leave an advisory `PENDING` note without committing to a real order. Under normal (`ACTIVE`) operation nothing is ever left `PENDING`, so `GET /disposition-recommendations/pending` will normally return an empty list.
- **Agent tool:** `fleet_tools.create_disposition_order_autonomous`.
- **Frontend:** `requestDispositionChannel()` creates the order directly. The lifecycle page's "Paused-agent advisory notes" section only shows anything if you've simulated a paused-agent fallback; it's empty under normal operation.

FLT-008/009 — Floorplan Covenant Risk

- **Decision:** `safe` / `approaching` (≥80% of `max_utilization_pct`) / `breached` (utilization strictly exceeds `max_utilization_pct`).
- **Autonomous action:** `breached` → `FloorplanFacility.frozen = True` immediately, blocking **all further pledges** against that facility (`pledge_vehicle` raises `FacilityFrozenError`, mapped to HTTP 423 Locked). `approaching` and `safe` take no blocking action.
- **Function:** `floorplan_engine.auto_freeze_facility(db, tenant_id, facility_id, reason, agent_key)`
- **Wired at:** `POST /api/v1/fleet/floorplan-facilities/{id}/covenant-risk-assessments`
- **The one human-only exception in this whole system:** `floorplan_engine.unfreeze_facility(db, tenant_id, facility_id)`, exposed at `POST /api/v1/fleet/floorplan-facilities/{id}/unfreeze`. No agent path calls this. This is intentional — see §2.

- **Deterministic arithmetic:** `is_covenant_breached`, `utilization_pct`, and every dollar figure on `FloorplanFacility` / `VinCollateralStatus` are exactly the auditable arithmetic they need to be for a lender. The AI only decides the risk *label* and whether to freeze; it never computes or touches the numbers themselves.
- **Frontend:** facility cards show a 🔒 FROZEN banner with an "Unfreeze (human action)" button (`unfreezeFacility(facilityId)`) when frozen. `pledgeCollateral()` checks `facility.frozen` first and refuses with an explanatory message if so.

FLT-010 — Driving Safety Scoring

- **Decision:** 0–100 score, floored at 0, recalculated automatically every 5th logged driving event (`REASSESSMENT_EVENT_INTERVAL`). Weights: harsh braking −5, rapid acceleration −5, speeding −10, phone usage −15, forward collision warning −20.
- **Autonomous action:** none beyond the scoring itself — this capability was already advisory-only by nature (there's no obvious downstream action a low score should trigger beyond visibility on the watchlist). **Not changed in this pass.**

FLT-011 — Fuel/Energy Anomaly *(chains into FLT-004)*

- **Decision:** `normal` / `flagged` / `not_evaluated`, from a plausibility-band check (MPG or mi/kWh, plus cost-per-unit) and, from the 2nd entry on, a running-average deviation check (40% threshold).
- **Autonomous action:** `flagged` → immediately opens a real `MaintenanceAlert` (`severity=PREDICTIVE`, `reason` includes the anomaly's own reasoning text).
- **Function:** `fuel_engine.auto_execute_fuel_anomaly(db, tenant_id, vehicle_id, assessment, reasoning)`
- **Wired at:** `POST /api/v1/fleet/fuel-entries/{id}/anomaly-assessments`
- **Important nuance:** the very first fuel entry for a vehicle has no running average to compare against, so only the plausibility-band check applies — missing history is never treated as a red flag by itself.

FLT-012 — Battery Degradation Forecast *(chains into FLT-004)*

- **Decision:** linear projection of days until SOH crosses the 80% alert threshold, computed from 2+ health readings.
- **Autonomous action:** if the projection is within `DEGRADATION_URGENCY_WINDOW_DAYS` (30 days) of crossing, immediately opens a real `MaintenanceAlert` — proactively, before the existing *reactive* single-reading threshold check (`isDegradationAlert` / SOH < 80%) would fire on its own.
- **Function:** `ev_charging_engine.auto_execute_battery_degradation_forecast(db, tenant_id, vehicle_id, estimated_days_to_threshold, reasoning)`
- **Wired at:** `POST /api/v1/fleet/vehicles/{id}/battery-degradation-forecasts`
- **Deliberately separate:** the reactive SOH < 80% watchlist check is fixed deterministic arithmetic — this forecast is additive to it, not a replacement.

FLT-013 — Fleet Optimization Capstone *(the other end of the FLT-006 chain, and the source of a second chain into FLT-007)*

- **Decision:** `retain` / `review_for_disposition` / `prioritize_disposition`, from a 0–3 point score: disposition-eligible (+1), availability < 85% (+1), latest valuation flagged (+1).
- **Autonomous action:** `prioritize_disposition` → fetches the vehicle's current odometer/class (`fleet_tools.get_vehicle`) and calls `evaluate_disposition_channel(...)` **directly, in-process**, which (per FLT-007's own autonomous execution above) creates a real `DispositionOrder`. `retain` and `review_for_disposition` take no further action.
- **Function:** `fleet_intelligence_agent.evaluate_fleet_optimization(...)`, new `chained_disposition` field on `FleetOptimizationResult`.
- **Failure isolation:** the chain call is wrapped in try/except — if the chained disposition-channel evaluation fails for any reason, the fleet-optimization decision itself is still recorded successfully. A failed chain step never blocks or corrupts the parent decision; it just means the disposition step didn't fire this time and can be requested again.

---

5. The full FLT-006 → FLT-013 → FLT-007 chain

This is the one multi-hop autonomous chain in the system, and it's worth drawing out explicitly since it spans three capabilities and two services (fleet-svc publishes the event; ai-orchestration-svc's Python function chains the last two hops in-process):

```
1. Market valuation recorded (fleet-svc)
        │
        ▼
2. FLT-006 evaluates: is deviation from book value ≥ 20%?
        │  FLAGGED
        ▼
3. fleet-svc's depreciation.py endpoint gathers TCO/availability/eligibility
   signals and publishes flt.fleet_optimization_requested (Kafka)
        │
        ▼
4. ai-orchestration-svc's evaluate_fleet_optimization runs FLT-013's
   0-3 point scoring
        │  prioritize_disposition (2-3 points)
        ▼
5. evaluate_fleet_optimization calls evaluate_disposition_channel
   directly (same process, no event round-trip needed for this hop)
        │
        ▼
6. FLT-007 picks the cascade channel from mileage and calls
   create_disposition_order_autonomous
        │
        ▼
7. Real DispositionOrder exists. Vehicle.status = REMARKETING.
   No human touched any of this.
```

Verification: this exact chain is exercised end-to-end in the frontend smoke test (`smoke_test.js`, the `"FLT-013: chain fires end-to-end on a fresh eligible+flagged vehicle"` check) by constructing a fresh eligible vehicle, flagging its valuation, and asserting a real disposition order comes out the other end. On the backend, each hop is tested individually (the chain logic itself is straightforward enough — a direct function call plus a try/except — that it doesn't need a dedicated integration test beyond what the unit tests for each hop already cover; consider adding one if this chain becomes more complex).

---

6. Schema changes

New migration: `f1a9c3d7e802_add_autonomous_execution_fields.py`

`Revises: a7dd087e6a37` (previous head — the FLT-013 capstone migration). This is now the head of the chain.

```
def upgrade() -> None:
    op.add_column('vehicles', sa.Column('held_for_review', sa.Boolean(),
nullable=False, server_default=sa.false()))
    op.add_column('vehicles', sa.Column('held_for_review_reason', sa.Text(),
nullable=False, server_default=''))
    op.add_column('vehicles', sa.Column('flagged_for_investigation',
sa.Boolean(), nullable=False, server_default=sa.false()))
    op.add_column('floorplan_facilities', sa.Column('frozen', sa.Boolean(),
nullable=False, server_default=sa.false()))
    op.add_column('floorplan_facilities', sa.Column('frozen_reason',
sa.Text(), nullable=False, server_default=''))
```

All five columns are additive with server defaults — this is a zero-downtime migration, safe to apply while the service is running. `downgrade()` drops them in reverse order.

Verified: applied cleanly through the full 15-migration chain (14 prior + this one) on a fresh SQLite database, and separately re-verified by extracting the delivered zip fresh and re-running `alembic upgrade head` from scratch.

Model docstring updates (no schema impact, but read these if you're new to the file)

- `TransferOrder` — docstring now describes the autonomous `auto_dispatch_transfer` path alongside the still-available manual `propose_transfer`/`dispatch_transfer` path.
- `DispositionChannelRecommendation` — docstring now describes `create_disposition_order_autonomous` as the agent's normal path, with the `PENDING`/accept/reject path called out as the paused-agent fallback only.
- `FloorplanFacility` — docstring now explains `frozen`/`frozen_reason` and that unfreezing is human-only.
- `Vehicle` — docstring now explains `held_for_review` and `flagged_for_investigation` as real operational holds, not advisory annotations.

7. API endpoint reference

Endpoint	Behavior
`POST /fleet/transfer-orders`	Creates a `PROPOSED` transfer order for manual dispatch. Available for non-agent use; the agent does not call this.
`POST /fleet/transfer-orders/auto-dispatch`	The agent's staging path — creates and dispatches a transfer atomically.
`POST /fleet/intake-inspections/{id}/readiness-assessments`	Records the readiness verdict and executes it in the same call (auto-provision or hold).
`POST /fleet/geofence-alerts/{id}/anomaly-assessments`	Records the anomaly verdict and executes it in the same call (auto-pull to maintenance on NEEDS_INVESTIGATION).
`POST /fleet/maintenance-triage-assessments`	Records the priority order and auto-dispatches a work order for the top-priority alert.
`POST /fleet/incidents/{id}/severity-assessments`	Records the severity verdict and auto-opens a repair order on SEVERE.
`POST /fleet/vehicles/{id}/disposition-recommendations`	Creates a real disposition order directly, writing the recommendation as `ACCEPTED`.
`POST /fleet/disposition-recommendations/{id}/accept`	Fallback path: turns a `PENDING` advisory note (left by a paused agent) into a real order. Not called by an active agent.
`POST /fleet/disposition-recommendations/{id}/reject`	Fallback path, same context as above.
`POST /fleet/floorplan-facilities/{id}/covenant-risk-assessments`	Records the risk level and auto-freezes the facility on BREACHED.
`POST /fleet/floorplan-facilities/{id}/unfreeze`	Human-only. Clears a freeze after a covenant breach is resolved.
`POST /fleet/asset-valuations/{id}/anomaly-assessments`	Records the valuation-anomaly verdict and, on FLAGGED, chains into a FLT-013 fleet-optimization request.
`POST /fleet/fuel-entries/{id}/anomaly-assessments`	Records the fuel/energy anomaly verdict and auto-opens a maintenance alert on FLAGGED.
`POST /fleet/vehicles/{id}/battery-degradation-forecasts`	Records the degradation forecast and auto-opens a maintenance alert if the projection is urgent.

Endpoints that both record and execute return the execution outcome in the response body (e.g. `executed: bool`, `auto_maintenance_alert_id`, `auto_repair_order_id`, `auto_frozen`, `chained_to_fleet_optimization` — field names vary per endpoint, check the handler).

8. Agent-side (ai-orchestration-svc) reference

Tool names

The staging and disposition-channel capabilities' Anthropic tool schemas and corresponding `fleet_tools.py` functions are named for what they execute, not what they recommend:

Capability	Tool / function name
FLT-003 Staging	`auto_dispatch_transfer`
FLT-007 Disposition	`create_disposition_order_autonomous`

If you're wiring up any external tooling, eval harness, or prompt cache against this agent, use these names — they are both the Python function name in `fleet_tools.py` and the Anthropic tool-schema `name` field the live-Claude path calls.

Result objects

- `StagingRecommendationResult.dispatched_transfer` — the transfer-order payload from `auto_dispatch_transfer`, or `None` if no transfer was warranted.
- `FleetOptimizationResult.chained_disposition` — set when a `prioritize_disposition` verdict triggers the FLT-007 chain; `None` otherwise, including when the chain isn't triggered or fails.

`DecisionLog.action_taken` values

Each `evaluate_*` function writes one of these into its `DecisionLog` row:

- `"transfer_dispatched"` / `"no_transfer_needed"` (staging)
- `"disposition_order_created"` (disposition channel)
- `"valuation_anomaly_recorded"` — check the fleet-svc response's `chained_to_fleet_optimization` field (not stored in this log) to see whether the FLT-013 chain fired
- `"fleet_optimization_recorded"` — check `chained_disposition` on the result object to see whether the FLT-007 chain fired
- `"skipped_agent_not_active"` — written by every capability when `Agent.status != ACTIVE`

System prompts

Every system prompt in `fleet_intelligence_agent.py` states plainly that the tool call executes the action, names the specific state change, and — where a downstream chain exists — describes it, so the live-Claude reasoning path accounts for it. Match this pattern for any new

capability: don't write a prompt that hedges toward "propose" or "recommend" language if the underlying tool actually executes.

---

9. Testing

Backend coverage

fleet-svc (`tests/test_api_remaining_screens.py`) covers the disposition-recommendation endpoint with:

- `test_disposition_recommendation_creates_real_order_autonomously` — asserts a real order + `REMARKETING` status come back from a single call, and that nothing is left pending
- `test_disposition_recommendation_autonomous_channel_choice_honored`
- `test_disposition_recommendation_creates_distinct_orders_each_call` — documents that there's deliberately no double-submission guard; two calls make two orders, since idempotency (if wanted) belongs at the caller/event layer, not as an artificial approval gate
- `test_paused_agent_advisory_fallback_still_available` — exercises the `PENDING`-only engine function directly, confirming the fallback path works for a paused agent even though the normal API route never leaves anything pending

ai-orchestration-svc (`tests/test_fleet_intelligence_agent.py`, `tests/test_api_fleet_intelligence.py`) covers every capability's deterministic-fallback path, live-Claude tool-call path, and paused-agent path, using the tool and field names from §8 throughout.

Two defensive details worth knowing if you extend this code:

1. `telematics_engine.py` imports `VehicleStatus`, required by `auto_execute_telemetry_anomaly`, which sets `Vehicle.status = MAINTENANCE`. Any new function in this file that touches vehicle status needs this import present.
2. `auto_dispatch_top_priority_work_order` validates `priority_order[0]` as a real UUID with a `try/except` before querying — a triage payload can in principle contain a stale or malformed alert ID, and this function skips (returns `None`) rather than raising in that case. The triage record itself is always written regardless.

Frontend coverage — `smoke_test.js`

72 assertions via jsdom, covering every capability's autonomous action plus the full FLT-006→013→007 chain. Run it after any change to `voyqi-fleet-management.html`:

```
cd /path/to/fleet-ui
node smoke_test.js
```

It extracts the `<script>` block, mocks `window.Chart`, evals it into a jsdom window, and drives the UI functions directly (form field values + button-handler calls) rather than simulating clicks — faster and just as meaningful for this codebase's plain-JS, no-framework structure.

A completeness check worth applying to any new capability: confirm every branch of its deterministic function is actually reachable through the UI. `hold_for_review` in FLT-001 required a genuine "Fail inspection" action (`inspectPO(id, passed)` takes a `passed` argument) — without it, that branch exists in code but nothing in the UI can trigger it.

Running everything

```
# Backend (from repo root, each service has its own venv/deps)
cd services/fleet-svc && python -m pytest tests/ -q
cd services/ai-orchestration-svc && python -m pytest tests/ -q
cd services/hrm-svc && python -m pytest tests/ -q
cd services/subscription-svc && python -m pytest tests/ -q
cd services/hub-svc && python -m pytest tests/ -q
# Expect: 197 / 374 / 208 / 70 / 38 = 887 total, all passing

# Migration check
cd services/fleet-svc
DATABASE_URL="sqlite+aiosqlite:///./test.db" python -m alembic upgrade head
# Expect: applies cleanly, 15 migrations, ends at f1a9c3d7e802

# Frontend
node smoke_test.js
# Expect: 72/72 PASS, "ALL CHECKS PASSED"
```

---

10. Deployment notes

- **Migration is additive and zero-downtime.** No backfill needed — new columns default to `false`/`''` for existing rows, which is the correct semantic (no existing vehicle is retroactively "held for review," no existing facility is retroactively "frozen").
- **No API surface removed.** `POST /transfer-orders`, the disposition accept/reject endpoints, and `dispatch_transfer` all exist and work for manual/non-agent use or the paused-agent fallback — the agent itself simply doesn't call them by default. Any other system integrating against those endpoints directly is unaffected.
- **Kafka event contract:** `flt.fleet_optimization_requested` can be published from either `depreciation.py` (the FLT-006 chain trigger) or `analytics.py`'s request endpoint, always with the same event shape (the `gather_fleet_optimization_signals` payload).
- **Rollback:** `alembic downgrade -1` drops the five new columns cleanly. Code rollback would need to happen alongside it, since the new engine functions reference those columns — don't downgrade the schema while the new code is still running.

---

11. File manifest

fleet-svc

```
src/models/__init__.py                    Vehicle + FloorplanFacility fields,
docstrings
src/services/staging_engine.py            auto_dispatch_transfer
src/services/inventory_engine.py          auto_execute_intake_readiness
src/services/telematics_engine.py         auto_execute_telemetry_anomaly (+
import fix)
src/services/maintenance_engine.py
auto_dispatch_top_priority_work_order
src/services/incident_engine.py           auto_execute_incident_severity
src/services/remarketing_engine.py        create_disposition_order_autonomous
src/services/floorplan_engine.py          auto_freeze_facility,
unfreeze_facility, FacilityFrozenError
src/services/fuel_engine.py               auto_execute_fuel_anomaly
src/services/ev_charging_engine.py
auto_execute_battery_degradation_forecast
src/api/staging.py                        POST /transfer-orders/auto-dispatch
src/api/inventory.py                      readiness-assessments now executes
src/api/telematics.py                     anomaly-assessments now executes
src/api/maintenance.py                    triage-assessments now executes
src/api/incidents.py                      severity-assessments now executes
src/api/remarketing.py                    disposition-recommendations now
creates real order
src/api/floorplan.py                      covenant-risk-assessments now
freezes; + unfreeze endpoint
src/api/depreciation.py                   anomaly-assessments now chains to
FLT-013
src/api/fuel.py                           anomaly-assessments now chains to
FLT-004
src/api/ev_charging.py                    battery-degradation-forecasts now
chains to FLT-004
alembic/versions/f1a9c3d7e802_*.py        new migration (head)
tests/test_api_remaining_screens.py       covers the disposition-
recommendation autonomous flow
```

ai-orchestration-svc

```
src/services/fleet_tools.py               auto_dispatch_transfer,
create_disposition_order_autonomous
src/services/fleet_intelligence_agent.py  every evaluate_* touched;
chained_disposition; system prompts
src/api/fleet_intelligence.py             dispatched_transfer field rename
tests/test_fleet_intelligence_agent.py    covers all 13 evaluate_* functions
and their chains
tests/test_api_fleet_intelligence.py      covers the AI-orchestration API
layer over those functions
```

Frontend

```
voyqi-fleet-management.html    all 13 modules' UI updated to match backend
autonomy;
```

```
                                inspectPO(id, passed) signature change + new
Fail button;
                                dispatchTransfer(),
acceptChannelRecommendation(),
                                rejectChannelRecommendation() removed
smoke_test.js                  72-assertion jsdom regression suite
```

---

12. If you're extending this system

1. **New capability, same pattern:** deterministic decision function → autonomous execution function in the relevant engine → wire into the API endpoint the agent already calls → update the system prompt to say plainly that the call executes the action → write a test that asserts the *state change*, not just the recorded assessment.
2. **New chain:** follow the FLT-006→013→007 pattern — either an in-process function call (cheap, same service) or a published Kafka event (cross-service, or if you want the chain step to be independently retryable/observable). Wrap chain calls in try/except so a failed downstream step never corrupts the upstream decision that's already been recorded.
3. **If something should NOT be autonomous:** say so explicitly in the docstring and system prompt, the way `unfreeze_facility` and `assess_liability` do. "This field has no AI path, structurally" is a stronger and more auditable statement than a comment saying "human should review this" on an otherwise-automatable function — structural separation is what a test like `test_incident_severity_never_calls_any_liability_related_function` can actually verify.
4. **Run the full matrix in §9 before considering any change complete.** A missing import or an unreachable UI branch is easy to write correctly on paper and only surfaces by actually running the tests and the smoke suite end-to-end — reading the code isn't enough.